

# Rockchip RK3576 USB Developer Guide

---

ID: RK-SM-YF-C01

Release Version: V1.2.0

Release Date: 2025-01-16

Security Level: ☐Top-Secret ☐Secret ☐Internal ☒Public

## DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD. ("ROCKCHIP") DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

## Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

**All rights reserved. ©2025. Rockchip Electronics Co., Ltd.**

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian, PRC

Website: [www.rock-chips.com](http://www.rock-chips.com)

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: [fae@rock-chips.com](mailto:fae@rock-chips.com)

## Preface

### Overview

This document provides a development guide for the RK3576 USB module, aimed at helping developers understand the hardware circuit design and software DTS configuration of the RK3576 USB controller and PHY, so that developers can flexibly design and develop quickly according to the USB application needs of the product.

| Chipset | Kernel Version |
|---------|----------------|
| RK3576  | Linux-6.1      |

### Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

| Version | Author                            | Date       | Change Description                                                                                                        |
|---------|-----------------------------------|------------|---------------------------------------------------------------------------------------------------------------------------|
| V1.0.0  | Wu Liangfeng<br>Wang<br>Mingcheng | 2024-04-24 | Initial version                                                                                                           |
| V1.0.1  | Wu Liangfeng                      | 2024-05-30 | Correction of usbdp_phy node name                                                                                         |
| V1.1.0  | Wu Liangfeng                      | 2024-10-09 | Correction of USB controller node name<br>Modification of Type-C controller chip support list, adding support for AW35615 |
| V1.2.0  | Wu Liangfeng                      | 2025-01-16 | Complete the DTS configuration description of Type-C USB 2.0 only                                                         |

# Contents

## Rockchip RK3576 USB Developer Guide

1. RK3576 USB Controller and PHY Introduction
2. RK3576 USB Config Map
3. RK3576 USB Hardware Circuit Design
  - 3.1 USB Controller Power Supply and Power Consumption Management
  - 3.2 USB PHY Power Supply and Power Consumption Management
    - 3.2.1 USB 2.0 PHY Power Supply and Power Consumption Management
    - 3.2.2 USB 3.1 PHY Power Supply and Power Consumption Management
      - 3.2.2.1 USB 3.1/DP Combo PHY
      - 3.2.2.2 USB 3.1/PCIe/SATA Combo PHY
  - 3.3 USB Hardware Circuit Design
    - 3.3.1 USB2\_OTG0\_VBUSDET Circuit Design
    - 3.3.2 Maskrom USB Circuit Design
    - 3.3.3 USB2 Circuit Design Supporting System Wake-Up
    - 3.3.4 Type-C USB 3.1/DP Full-Function Hardware Circuit
    - 3.3.5 Type-C to Type-A USB 3.1/DP Hardware Circuit
    - 3.3.6 Type-C to Type-A/Micro USB 2.0/DP Hardware Circuit
    - 3.3.7 Type-A USB3.1 OTG1 Hardware Circuit
4. RK3576 USB DTS Configuration
  - 4.1 USB Chip-Level DTSI Configuration
  - 4.2 USB 3.1/DP Full-Function DTS Configuration
  - 4.3 USB 3.1/DP DTS Configuration for Type-C to Type-A
  - 4.4 Type-C to Type-A/Micro USB 2.0/DP DTS Configuration
  - 4.5 Type-C USB 2.0 only DTS Configuration
  - 4.6 USB 3.1 DTS Configuration for Type-A
  - 4.7 DTS Configuration for USB PHY Without Power Supply
    - 4.7.1 USBDP PHY DTS Configuration Without Power Supply
    - 4.7.2 ComboPHY1 Powerless DTS Configuration
  - 4.8 Considerations for Configuring Linux USB DT
    - 4.8.1 Important Properties of USB DT
      - 4.8.1.1 USB Controller Attributes
      - 4.8.1.2 USB2 PHY Attributes
      - 4.8.1.3 USBDP Combo PHY Properties
      - 4.8.1.4 USBC Properties
5. RK3576 USB OTG Mode Switching Commands
6. Type-C Controller Chip Support List
7. Reference Documents

# 1. RK3576 USB Controller and PHY Introduction

RK3576 supports 2 independent USB 3.1 OTG controllers, 2 independent USB 2.0 PHYs, 1 USB 3.1/DP Combo PHY, and 1 USB 3.1/SATA/PCIe Combo PHY. For more detailed characteristics of the USB controller, please refer to the RK3576 TRM.

The RK3576 USB has added the following features:

- 1. Both OTG0/1 support MMU, allowing USB controller hardware to access memory space beyond 4G;
- 2. OTG1 supports CCI to ensure Cache coherence, meaning there is no need for the CPU to perform cache flushing operations (OTG0 does not support this);
- 3. OTG0 DP/DM supports multiplexing with UART/JTAG, and the SDK enables this function by default, which requires the use of the RK USB to DEBUG adapter board.

**Note:**

- 1. The USB 3.1 Gen1 physical layer transfer rate is 5Gbps, and the USB 2.0 physical layer transfer rate is 480Mbps;
- 2. The USB 3.1/DP Combo PHY supports 4 lanes, which can simultaneously support USB 3.1 + DP 2 lanes;
- 3. Usage restriction: The USB 3.1/SATA/PCIe Combo PHY can only support one working mode at a time, meaning that the USB 3.1 interface is mutually exclusive with the SATA/PCIe interface.

Table 1 Connection Relationship of RK3576 USB Controller and PHY

| USB Interface Name<br>(Schematic) | USB Controller      | USB PHY                                  |
|-----------------------------------|---------------------|------------------------------------------|
| USB OTG0                          | OTG0<br>(DWC3&xHCI) | USB3.1/DP ComboPHY + USB2.0 PHY0         |
| USB OTG1                          | OTG1<br>(DWC3&xHCI) | USB3.1/SATA/PCIe ComboPHY1 + USB2.0 PHY1 |

The pin correspondence between the RK3576 USB controller and chip-side USB data transmission is shown in Table 2 below.

Table 2 Correspondence between RK3576 USB Controller and USB Pin Feet

| USB Controller/Pin Feet | RK3576 USB data pin                                                                                                                                                                |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| USB 3.1 OTG0            | USB2_OTG0_DP/USB2_OTG0_DM,<br>USB3_OTG0_SSRX1P/USB3_OTG0_SSRX1N,<br>USB3_OTG0_SSTX1P/USB3_OTG0_SSTX1N,<br>USB3_OTG0_SSRX2P/USB3_OTG0_SSRX2N,<br>USB3_OTG0_SSTX2P/USB3_OTG0_SSTX2N, |
| USB 3.1 OTG1            | USB2_OTG1_DP/USB2_OTG1_DM,<br>USB3_OTG1_SSTXP/USB3_OTG1_SSTXN,<br>USB3_OTG1_SSRXP/USB3_OTG1_SSRXN,                                                                                 |

The internal connection relationship of the RK3576 USB controller and PHY, as well as the corresponding common USB physical interfaces, is shown in Figure 1 below.

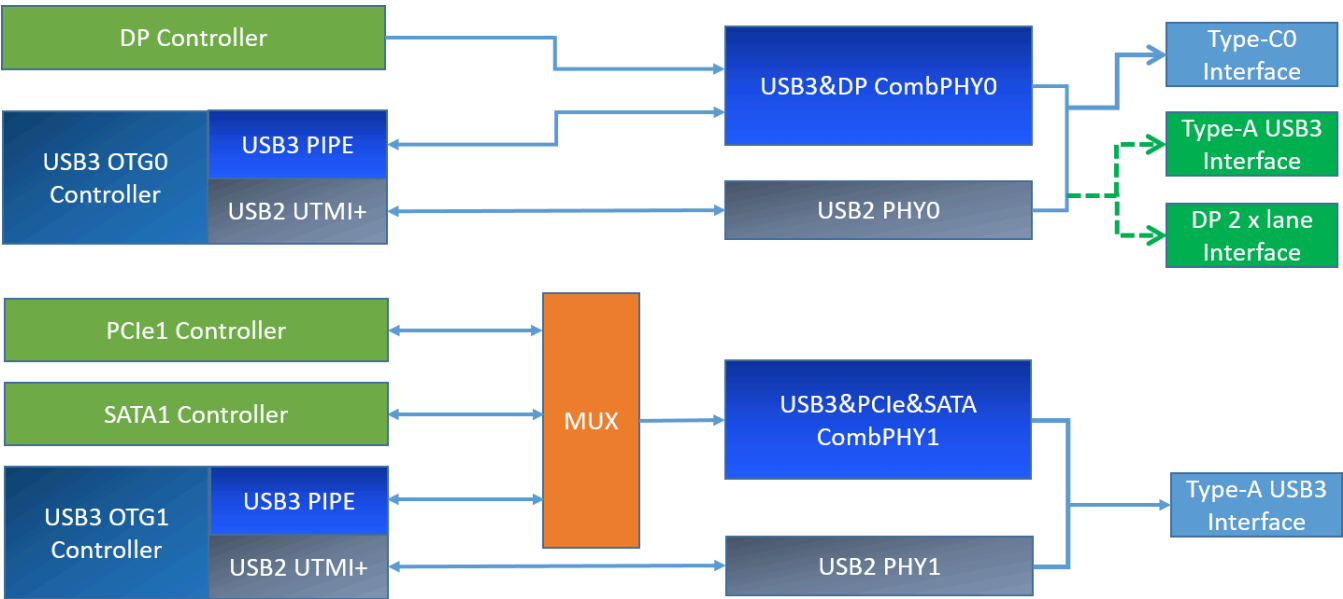


Figure 1 Schematic diagram of RK3576 USB controller and PHY connection

**Note:**

1. The types of interfaces supported by RK3576 USB are not limited to the Type-C/A USB interface types described in Figure 1, and can also support all common USB interfaces, including Type-C USB 2.0/3.1, Type-A USB 2.0/3.1, Micro USB 2.0/3.1, etc. To adapt to different USB circuit designs and interface types, the Linux-6.1 kernel USB driver has been software-compatible, and developers only need to correctly configure the Linux USB DTS according to the product's USB hardware circuit to enable the corresponding USB interface functions. For detailed USB DTS configuration methods, please refer to [RK3576 USB DTS Configuration](#).
2. **Special Usage Restrictions:** OTG1/PCIe1/SATA1 controller bus access is mutually exclusive; if PCIe1 or SATA1 has been used in the hardware circuit design, then the OTG1 USB2 and USB3 functions cannot be used.

## 2. RK3576 USB Config Map

The two independent USB controllers of RK3576 can support various product forms as shown in Figures 2 to 4 below.

### USB OTG0 can support 5 hardware circuit designs:

Config0: Type-C0 USB3.1 OTG0 with DP function

Config1: USB 2.0 OTG0 + DP 4 x Lane (Swap off)

Config2: USB 2.0 OTG0 + DP 4 x Lane (Swap on)

Config3: USB 3.1 OTG0 + DP 2 x Lane (Swap on)

Config4: USB 3.1 OTG0 + DP 2 x Lane (Swap off)

### USB OTG1 supports 2 hardware circuit designs:

Config0: USB 2.0 only OTG1

Config1: USB 3.1 OTG1

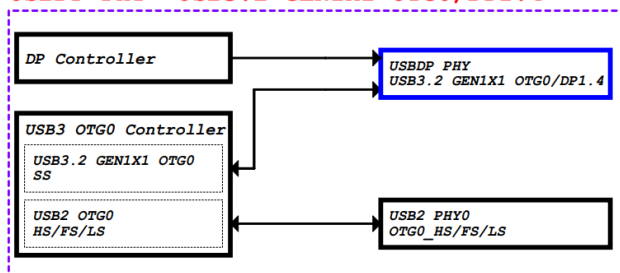
For a more detailed USB configuration map, please refer to the USB/DP Configure Map section of the RK3576 SDK EVB reference schematic.

### Note:

1. USB 2.0/3.1 OTG1 cannot be used simultaneously with PCIe1/SATA1.
2. The Lanes correspondence for USB DP Swap on/off configurations is as follows:  
Swap off: Lane0/1/2/3 TxData mapping to Lane0/1/2/3\_TXDP/N  
Swap on: Lane0/1/2/3 TxData mapping to Lane2/3/0/1\_TXDP/N

#### MULTI\_PHY Path Map

##### USB DP PHY--USB3.2 GEN1X1 OTG0/DP1.4



##### PCIe Combo PHY1-- PCIe1/SATA1/USB3.2 GEN1X1 OTG1

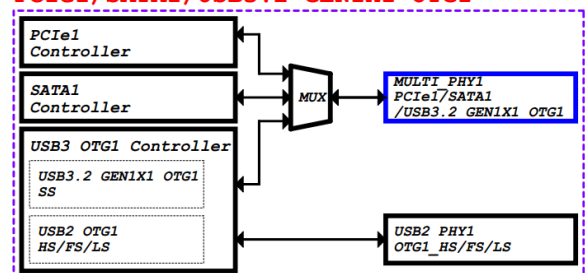
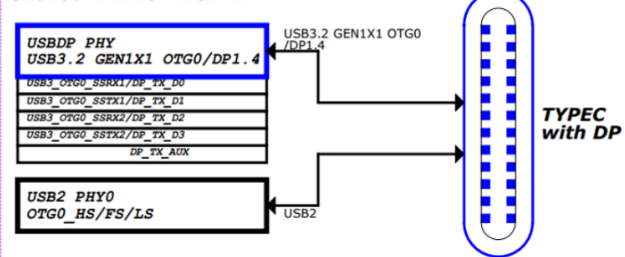


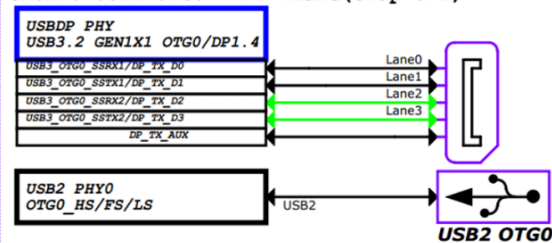
Figure 2 RK3576 USB Path Map

## USB OTG0/DP Application

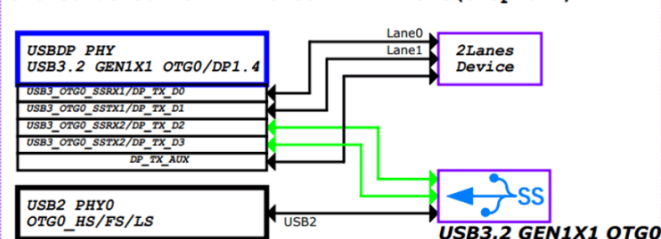
CASE0: TYPEC with DP



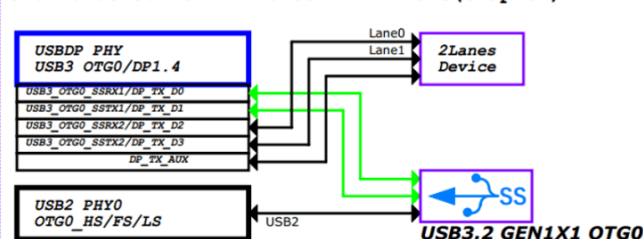
CASE1: USB2 OTG0 + DP 4Lane(Swap OFF)



CASE3: USB3.2 GEN1X1 OTG0 + DP 2Lane(Swap OFF)



CASE4: USB3.2 GEN1X1 OTG0 + DP 2Lane(Swap ON)



CASE2: USB2 OTG0 + DP 4Lane(Swap ON)

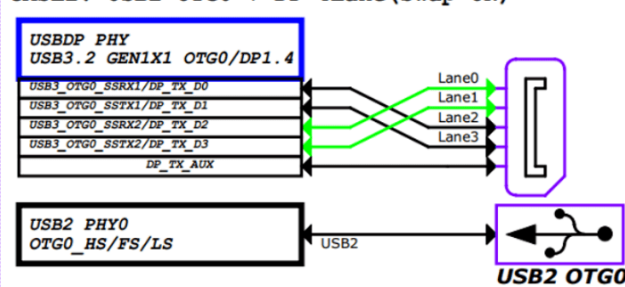
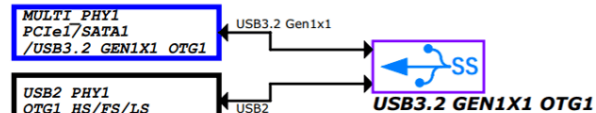


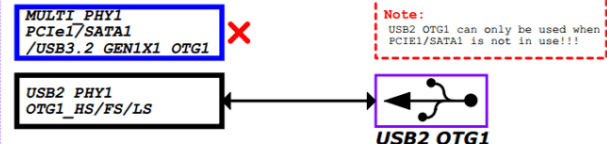
Figure 3 RK3576 OTG0 USBDP Path Map

## USB OTG1 Application

CASE0: USB3.2 GEN1X1 OTG1



CASE1: USB2 OTG1



CASE2: Do not support USB



Figure 4 RK3576 OTG1 MULTI PHY Path Map

## 3. RK3576 USB Hardware Circuit Design

This section primarily outlines the various hardware circuit design schemes that RK3576 USB can support in practical applications. The interfaces that RK3576 can support are as follows:

- USB30/DP1.4 MULTI0
- USB30/PCIE2.0/SATA30 MULTI1



## 3.1 USB Controller Power Supply and Power Consumption Management

The RK3576 USB controller is powered by VD\_LOGIC. Additionally, the chip has a dedicated power domain designed for the USB controller: USB3.1 OTG0 is located in PD\_USB; USB3.1 OTG1 is located in PD\_PHP.

In practical usage scenarios, the Linux USB controller driver will dynamically switch the PD of the USB controller based on the working condition of the USB interface, using the Linux PM Runtime mechanism to reduce the power consumption of the USB controller. When the system enters the second-level standby, to achieve optimal power consumption, the software will forcibly shut down all PDs of the USB controller. Therefore, in actual product application scenarios, if it is necessary to maintain the working state of the USB controller's registers during the second-level standby, the function `device_init_wakeup` needs to be called in the USB controller driver to prevent the PD of the USB controller from being shut down during the second-level standby.

**The power consumption control strategy for the USB controller is as follows:**

1. For unused USB controllers, the corresponding controller DTS node should be configured as disabled;
2. For USB controllers enabled by the kernel, the kernel USB driver already supports the Auto suspend feature of the USB controller (when the USB HOST interface is not connected to any external devices, the controller automatically enters a suspend low-power state), therefore, developers do not need to debug the dynamic power management of the USB controller.

## 3.2 USB PHY Power Supply and Power Consumption Management

### 3.2.1 USB 2.0 PHY Power Supply and Power Consumption Management

RK3576 supports two independent USB 2.0 PHYs. Internally, all USB 2.0 PHYs belong to VD\_USBPHY, and all USB 2.0 PHYs share the three external power supply sources as shown in Figure 5 below. Therefore, during system operation, it is not possible to reduce the power consumption of the USB 2.0 PHYs by simply turning off the hardware power and closing PD.

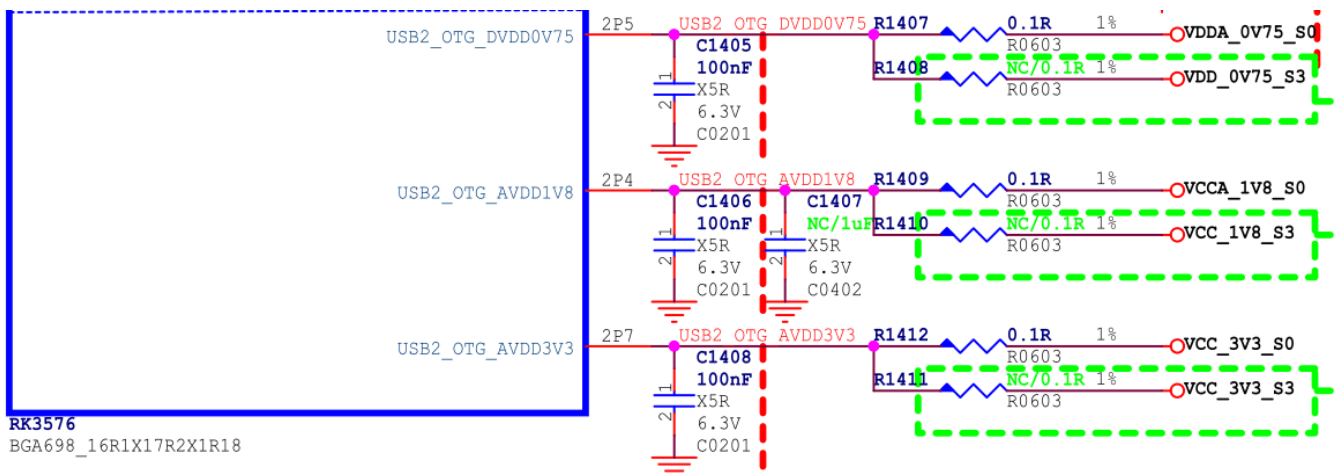


Figure 5 USB 2.0 PHY Power Supply

It should be noted that in actual circuits, if the supply voltage of the USB 2.0 PHY exceeds the specified maximum value or falls below the specified minimum value, it may cause abnormal USB connections.

Table 3 USB 2.0 PHY Supply Voltage Requirements

| Supply Voltage     | Minimum | Normal | Maximum | Unit |
|--------------------|---------|--------|---------|------|
| USB2_OTG_DVDD_0V75 | 0.6975  | 0.75   | 0.825   | V    |
| USB2_OTG_AVDD_1V8  | 1.674   | 1.8    | 1.98    | V    |
| USB2_OTG_AVDD_3V3  | 3.069   | 3.3    | 3.63    | V    |

**USB 2.0 PHY Power Consumption Control Strategy is as follows:**

1. To support the function of downloading firmware through Maskrom USB, it is necessary to ensure that the three power supplies of the USB 2.0 PHY are normal;
2. After the system is powered on, all USB 2.0 PHYs are in Normal mode by default. The software configures USB 2.0 PHY1/PHY2/PHY3 to the lowest power consumption IDDQ mode during the U-Boot SPL phase (SDK is already supported), and after entering the system, the kernel USB driver will set the corresponding USB 2.0 PHY to exit IDDQ mode according to the application requirements;
3. For the USB 2.0 PHY enabled by the kernel, the kernel USB 2.0 PHY driver will automatically perform dynamic power consumption control on the PHY. When a device is detected, it automatically sets the USB 2.0 PHY to Normal mode, and when no device is detected, it automatically sets the USB 2.0 PHY to Suspend mode;

The power consumption data of the USB 2.0 PHY in different working modes is shown in Table 4 below.

Table 4 USB 2.0 PHY Power Consumption Data (Statistics for the power consumption of a single USB 2.0 PHY)

| Supply Voltage  | Read/Write Data | Dynamic Suspend | PHY Disabled | Secondary Standby | Unit |
|-----------------|-----------------|-----------------|--------------|-------------------|------|
| USB20_DVDD_0V75 | 8.9             | 2.8             | 0.05         | 0                 | mA   |
| USB20_AVDD_1V8  | 8.6             | 3.34            | 0.05         | 0                 | mA   |
| USB20_AVDD_3V3  | 2.5             | 0.14            | 0.05         | 0                 | mA   |

**Note:**

1. The test scenario for power consumption of read/write data: Copying data with a U2 disk, PHY is in Normal mode;
2. The test scenario for dynamic suspend power consumption: USB 2.0 PHY DTS is enabled, but no USB peripheral is connected, PHY is in Suspend mode;
3. The test scenario for PHY disabled power consumption: USB 2.0 PHY DTS is disabled, PHY is in IDDQ mode;
4. The test scenario for secondary standby power consumption: All three power supply sources of the USB 2.0 PHY are turned off.

3.2.2 USB 3.1 PHY Power Supply and Power Consumption Management

RK3576 supports two types of USB 3.1 Combo PHYs:

- 1. USB 3.1/DP Combo PHY
- 2. USB 3.1/PCIe/SATA Combo PHY

The power supply and power consumption control methods for these two USB 3.1 Combo PHYs are different, and are described separately below.

3.2.2.1 USB 3.1/DP Combo PHY

RK3576 USB3.1 OTG0 utilizes the USB 3.1/DP Combo PHY. Internally within the chip, the USB 3.1/DP Combo PHY belongs to VD\_USBDPPHY (Alive), and externally, there are two independent power supply sources as shown in Figure 6 below.

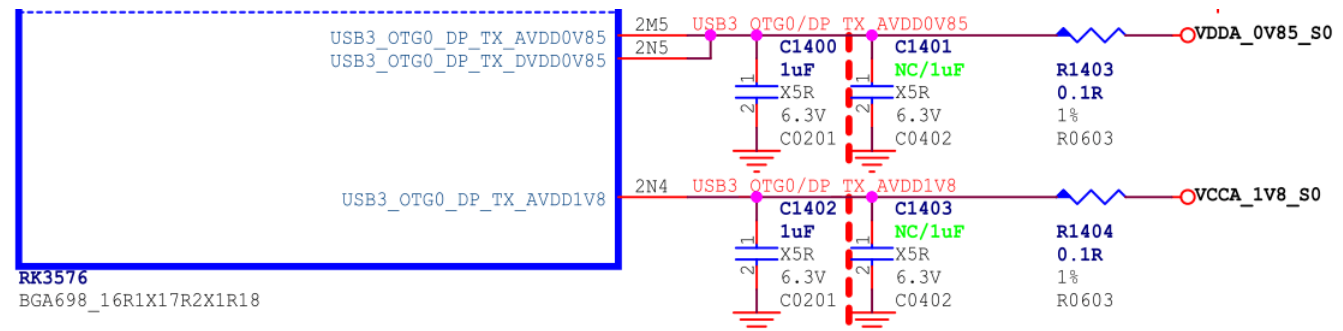


Figure 6 RK3576 USBDP Combo PHY Power Supply

Table 5 USB 3.1/DP Combo PHY Power Supply Voltage Requirements

| Power Supply       | Minimum | Normal | Maximum | Unit |
|--------------------|---------|--------|---------|------|
| VDD_0V85/VDDA_0V85 | 0.8075  | 0.85   | 0.8925  | V    |
| VDDH_1V8           | 1.71    | 1.8    | 1.89    | V    |

USB 3.1/DP Combo PHY Power Consumption Control Strategy is as follows:

- 1. After the system powers on, the USBDP PHY is in an uninitialized state with the lowest power consumption;
- 2. In scenarios supporting USBDP, the kernel USBDP PHY driver will automatically perform dynamic power consumption control on the PHY. When a device is detected, it automatically sets the USBDP PHY to P0 State. When no device is detected, it automatically sets the USBDP PHY to P3 State (for Type-A interfaces) or to reset state (for Type-C interfaces);
- 3. For scenarios not using USBDP (i.e., neither USB3.1 nor DP is used), the power supply of the USBDP PHY can be designed to either normally power or cut power according to project requirements. The details are as follows:
  - (1) If USB3.1 firmware download functionality is required, the power supply of the USBDP PHY must be normally powered;
  - (2) If USB3.1 firmware download functionality is not required, it is recommended to externally cut

power to the USBDP PHY's power supply, but the DTS configuration must be modified. For details, please refer to [USB PHY Power Supply Disabled DTS Configuration](#);

(3) If USB3.1 firmware download functionality is not required and the power supply of the USBDP PHY is normally powered, it is recommended to configure the USBDP PHY DTS node as disabled, which means the PHY is in an uninitialized state with the lowest power consumption;

Table 6 USB 3.1/DP Combo PHY Power Consumption Data

| Power Supply       | Read/Write Data | Dynamic Sleep | PHY Disabled | L2 Standby | Unit |
|--------------------|-----------------|---------------|--------------|------------|------|
| VDD_0V85/VDDA_0V85 | 101.6           | 5             | 2            | 0          | mA   |
| VDDH_1V8           | 29              | 0             | 0            | 0          | mA   |

**Note:**

1. The power consumption test scenario for read/write data: copying data from a U3 disk, PHY is in P0 state;
2. The power consumption test scenario for dynamic sleep: Type-C interface, no USB device connected, PHY is in reset state;
3. The power consumption test scenario for PHY Disabled: The DTS node of PHY is configured as disabled, PHY is in an uninitialized state, and in this state, the power consumption is the lowest;
4. The power consumption test scenario for L2 Standby: Both power supplies of the PHY are turned off;

Kernel method to disable USBDP PHY is as follows:

```
&usbdp_phy {
    status = "disabled";
};

&usbdp_phy_dp {
    status = "disabled";
};

&usbdp_phy_u3 {
    status = "disabled";
};
```

### 3.2.2.2 USB 3.1/PCIe/SATA Combo PHY

The RK3576 USB3.1 OTG1 utilizes a USB3.1/PCIe/SATA Combo PHY1. Internally within the chip, this PHY belongs to the PD\_BUS (Alive), and externally it has two independent power supply sources as shown in Figure 7.

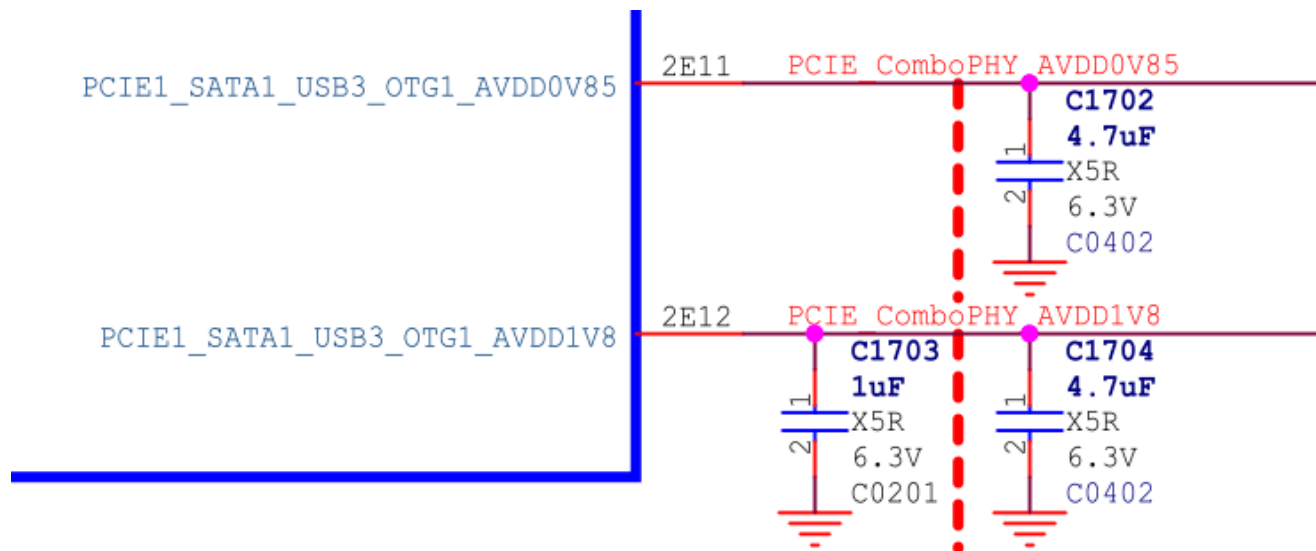


Figure 7 RK3576 USB 3.1/PCIe/SATA Combo PHY Power Supply

Table 7 USB 3.1/PCIe/SATA Combo PHY Voltage Supply Requirements

| Power Supply | Minimum | Normal | Maximum | Unit |
|--------------|---------|--------|---------|------|
| AVDD_0V85    | 0.8     | 0.85   | 0.935   | V    |
| AVDD_1V8     | 1.62    | 1.8    | 1.98    | V    |

**Power Consumption Control Strategy for USB 3.1/PCIe/SATA Combo PHY is as follows:**

1. When the chip is powered on, the USB 3.1/PCIe/SATA Combo PHY is in the working state by default. The software sets the PHY to reset state during the U-Boot SPL phase to maintain the lowest power consumption. After entering the kernel, the USB controller driver releases the PHY's reset by calling the `rockchip_combphy_init()` function.
2. Dynamic power consumption control of the PHY: When the Combo PHY operates in USB mode, the PIPE state (P0/P1/P2/P3) of the PHY is automatically controlled by the USB controller hardware, dynamically entering and exiting P0/P1/P2/P3 states according to different working scenarios. For example, when no USB device is inserted, the PIPE is in P3 state; when a U3 disk is inserted, it switches to P0 state; when a U3 HUB is connected, as long as the downstream port of the HUB does not connect to other USB peripherals, the PIPE state automatically enters P3 state for low power consumption. When a USB peripheral is inserted into the U3 HUB, the PIPE state automatically switches to P0. (Note: P0 is the normal working state, P3 is the lowest power consumption state)
3. When it is clear that the USB 3.1 OTG1/PCIe/SATA interface is not used, the corresponding Combo PHY can be unpowered, but the DTS needs to modify the configuration, please refer to [USB PHY Power Supply Not Enabled DTS Configuration](#);
4. When the PHY is powered, if this PHY is not used, the corresponding PHY DTS node needs to be configured as disabled, which means keeping the PHY in reset state with the lowest power consumption.

Table 8 USB 3.1/SATA/PCIe Combo PHY Power Consumption Data

| Power Supply | Read/Write Data | Dynamic Sleep | PHY Disabled | Second-Level Standby | Unit |
|--------------|-----------------|---------------|--------------|----------------------|------|
| AVDD_0V85    | 44.5            | 9.6           | 0.4          | 0                    | mA   |
| AVDD_1V8     | 5.2             | 0.5           | 0.2          | 0                    | mA   |

**Note:**

1. The power consumption test scenario for read/write data: Copying data with a U3 disk, PHY is in P0 state;
2. The power consumption test scenario for dynamic sleep: PHY DTS is enabled, but no USB peripheral is connected, PHY is in P3 State;
3. The power consumption test scenario for PHY disabled: The DTS node of PHY is configured as disabled, PHY is in reset state;
4. The power consumption test scenario for second-level standby: Both power supply sources of the PHY are turned off.

Method to disable USB 3.1/PCIe/SATA Combo PHY in the kernel is as follows:

```
&combphy1_psu {
    status = "disabled";
};
```

## 3.3 USB Hardware Circuit Design

### 3.3.1 USB2\_OTG0\_VBUSDET Circuit Design

USB2\_OTG0\_VBUSDET is used for detecting the connection and disconnection of devices when USB is operating as a Device. The design considerations are as follows:

1. For designs supporting PD functionality with a Type-C interface, i.e., supporting external Type-C controller chips (such as FUSB302 or HUSB311), refer to the circuit design of RK3576 EVB1 Type-C (USB2\_OTG0\_VBUSDET is fixedly pulled up to VCC\_3V3\_S0). The software driver can detect the connection and disconnection of USB Devices through the CC pins of the Type-C controller chip;
2. For other circuit design schemes without support for external Type-C controller chips (such as Type-C USB 2.0 only, Type-A USB 3.1, Micro USB 2.0/3.1), USB2\_OTG0\_VBUSDET should still be designed according to the traditional voltage divider circuit, connected to the VBUS pin of the USB interface. VBUSDET should not be designed as a constant power supply (if there is a need for USB HOST functionality, an independent GPIO or PMIC VBUS control is required, not sharing with other USB HOST VBUS control circuits);
3. The high-level voltage range of the chip input USB2\_OTG0\_VBUSDET is required to be **[0.9V ~ 3.3V]**.

### 3.3.2 Maskrom USB Circuit Design

Maskrom supports firmware download via USB3.1, while being backward compatible with USB2.0 firmware downloads. The hardware design requirements are as follows:

1. The power supply for USB 2.0 PHY must be powered, for details, please refer to [USB 2.0 PHY Power Supply and Power Consumption Management](#);
2. The power supply for USBDP PHY can be chosen between normal power supply or power off based on the actual project requirements, for details, please refer to [USB 3.1 PHY Power Supply and Power Consumption Management](#);

If USB3.1 firmware download is supported, the power supply for USBDP PHY must be powered, and USB3\_OTG0\_TX1/RX1 must be connected to the USB download port.

3. The enumeration of Maskrom USB does not depend on the level of USB2\_OTG0\_VBUSDET;
4. Firmware Writing Tool Configuration Instructions

(1) The functionality of USB 3.1 firmware download must be used in conjunction with Windows tool v3.28 or Linux tool v2.30 or newer versions, and manually modify the config.ini of the firmware writing tool.

Config.ini modification for Windows tool: USB3\_TRANSFER=TRUE

Config.ini modification for Linux tool: usb3\_transfer\_on=true

(2) In hardware circuit design, if only OTG0 USB2 only is supported and the USBDP PHY is not powered (i.e., USB3\_OTG0\_DP\_TX\_AVDD0V85, USB3\_OTG0\_DP\_TX\_DVDD0V85, USB3\_OTG0\_DP\_TX\_AVDD1V8 these three routes are not powered), the USB3 functionality of the firmware writing tool must be turned off, otherwise, it will cause the Maskrom firmware download to fail.

### 3.3.3 USB2 Circuit Design Supporting System Wake-Up

USB2 supports system wake-up through DP/DM, and this function has specific power supply requirements for the second-level standby mode, as detailed below:

1. The three external power supplies for USB2 PHY (USB2\_OTG\_DVDD\_0V75, USB2\_OTG\_AVDD\_1V8, USB2\_OTG\_AVDD\_3V3) must remain powered;
2. The VBUS of the USB interface must continue to supply power to USB peripherals;
3. The external power supply for RK3576's OSC (OSC\_AVDD1V8) must remain powered;
4. The external power supply for RK3576's PMU must remain powered;
5. The external power supply for RK3576's Logic can either be powered or disconnected. It is recommended to disconnect the Logic to reduce power consumption in the second-level standby mode.

### 3.3.4 Type-C USB 3.1/DP Full-Function Hardware Circuit

Taking the hardware circuit design of RK3576 EVB1 USB 3.1 OTG0 Type-C as an example.

1. The Type-C circuit must be paired with an external Type-C controller chip to achieve the full functionality of Type-C (including: detection of both sides, PD charging negotiation, Alternate Mode negotiation). RK3576 can support most commonly used Type-C controller chips, for details, please refer to the [Type-C Controller Chip Support List](#);
2. To support high-voltage charging functions while reducing the risk of hardware circuitry, [USB2\\_OTG0\\_VBUSDET](#) should not be connected to the VBUS of the Type-C interface, just fixedly pulled up to 3.3V (as shown in Figure 8 and Figure 9, USB2\_OTG0\_VBUSDET connected to VCC\_3V3\_S0), but must not be left floating;
3. USB2\_OTG0\_ID is only used for the OTG function of the Micro interface type, the Type-C circuit does not need to use it, just leave it floating.
4. TYPEC\_SBU1/TYPEC0\_SBU2 is only used for AUX communication in DP Alternate Mode. According to the protocol requirements of AUX, the SBU1/SBU2 should be pulled up to the corresponding level based on the front and back insertion of the Type-C. Since the RK3576 chip does not implement the automatic pull-up of SBU1/SBU2 internally, it is required to add two GPIO controls externally in the hardware circuit (corresponding to TYPEC\_SBU1/TYPEC\_SBU2 in Figure 9). There is no requirement for the default pull-up and pull-down method of GPIO, any GPIO can be chosen. In software, the properties `sbu1-dc-gpios` and `sbu2-dc-gpios` of the `usbdp_phy` node need to be modified for adaptation;
5. The software configuration of Type-C USB DTS, please refer to [Type-C USB 3.1/DP Full-Function DTS Configuration](#);

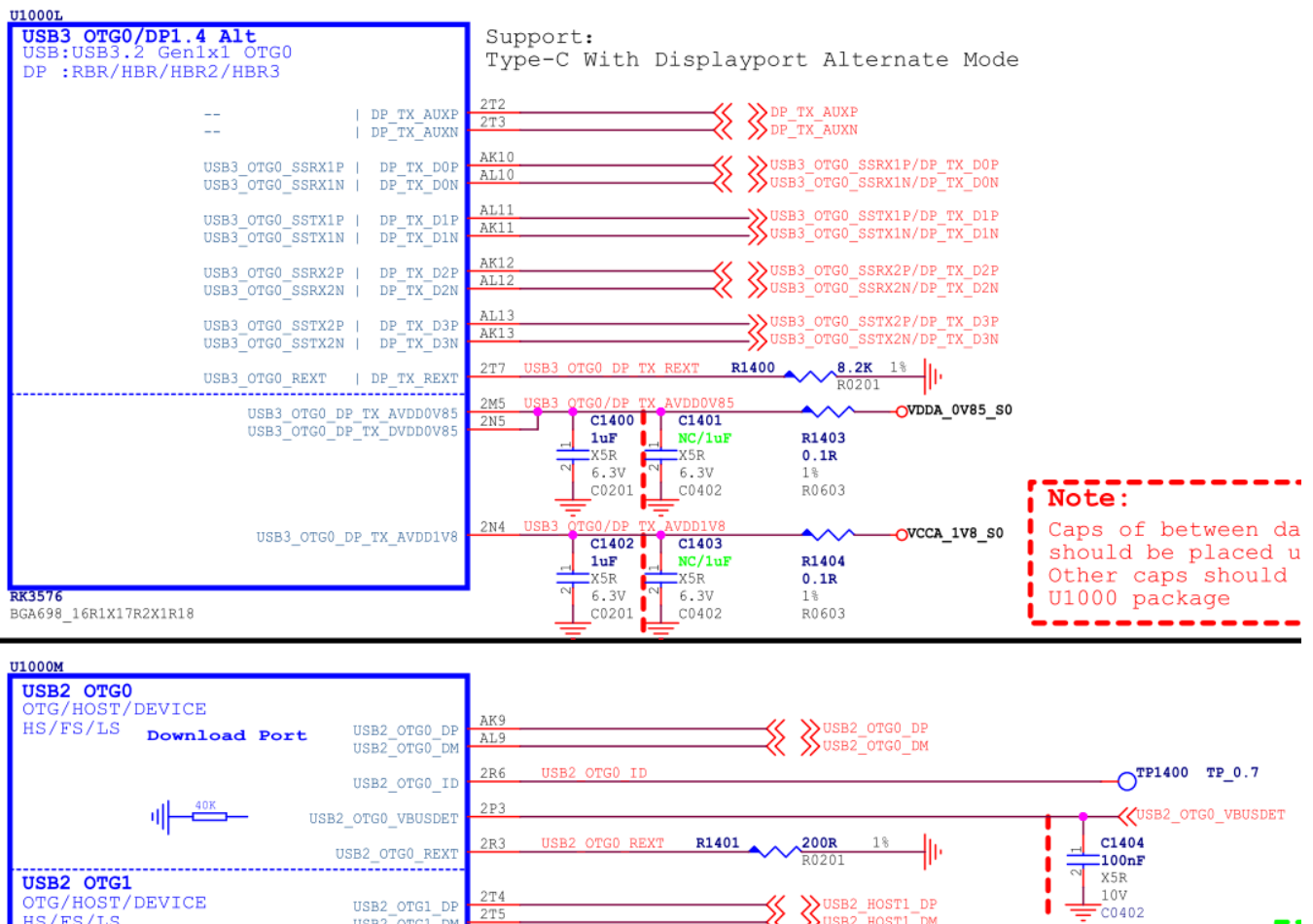




Figure 8 RK3576 Type-C Circuit

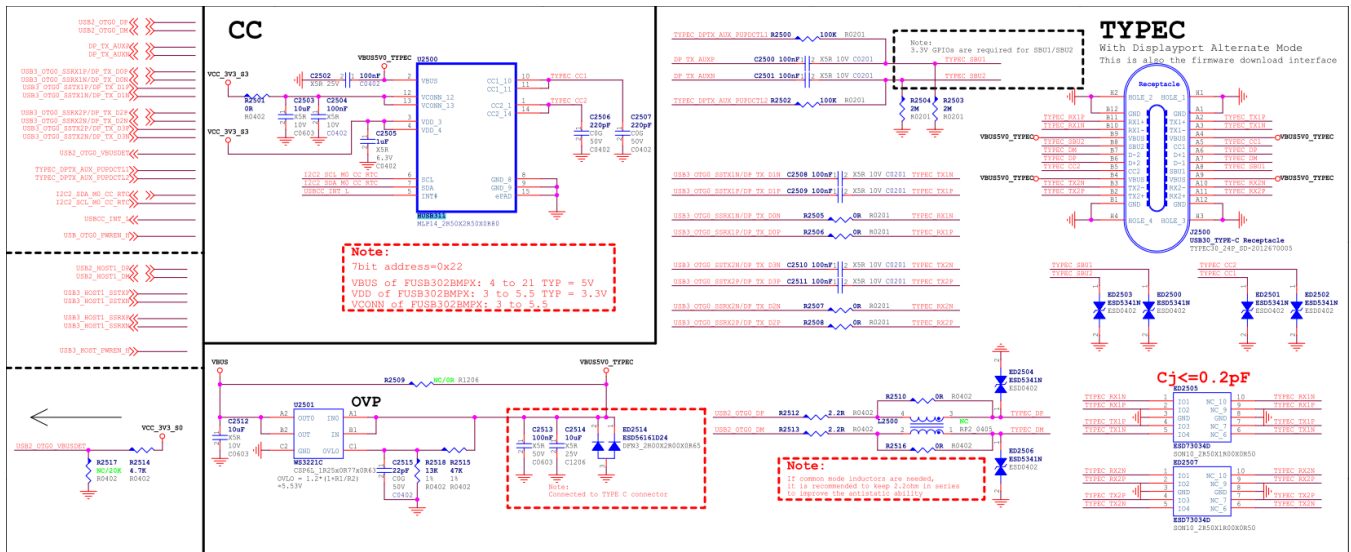


Figure 9 RK3576 Type-C Interface

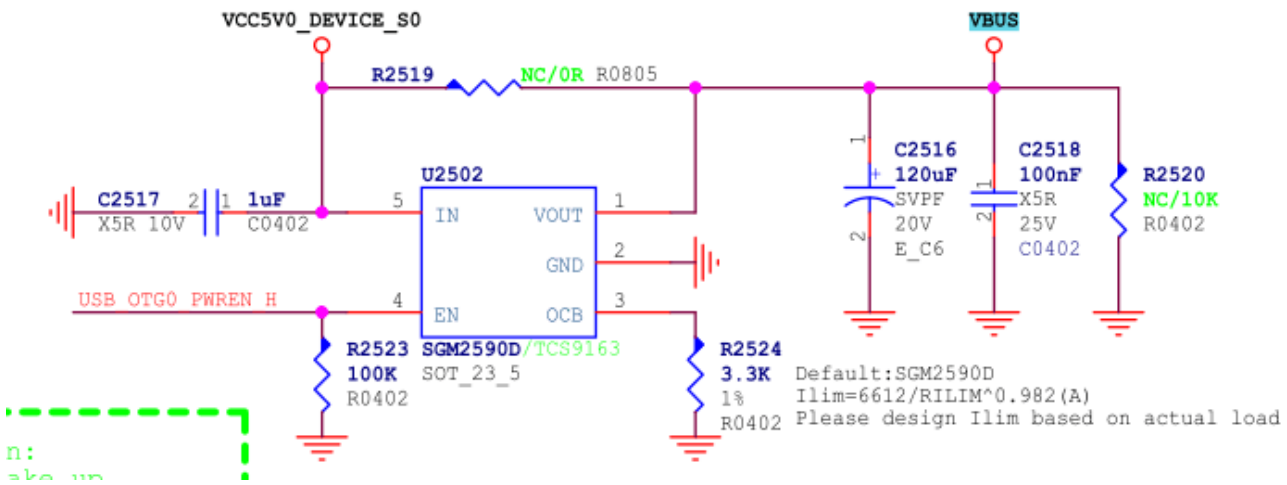


Figure 10 RK3576 Type-C VBUS Control Circuit

### 3.3.5 Type-C to Type-A USB 3.1/DP Hardware Circuit

This solution is applicable to RK3576 USB 3.1 OTG0 Type-C, which can be split into separate Type-A USB 3.1 interfaces and DP (2 x Lane) interfaces for use. Taking the hardware circuit design of RK3576 EVB2 Type-C to Type-A USB 3.1/DP as an example.

1. Type-A USB 3.1 utilizes USB3\_OTG0\_SSRX1P/N and USB3\_OTG0\_SSTX1P/N (corresponding to lane0/1 of the chip's internal USBDP PHY), while DP uses DP\_TX\_D2P/N and DP\_TX\_D3P/N (corresponding to lane2/3 of the chip's internal USBDP PHY);
2. If the USB OTG has scenarios where it is used as a Device/HOST, it is recommended that USB2\_OTG0\_VBUSDET be connected to the VBUS of the Type-A USB interface through a 30KΩ resistor;
3. The power supply for Type-A VBUS (VCC5V0\_USB30\_HOST0) is controlled by GPIO. When OTG is in Device mode, VBUS output is turned off. When OTG is in HOST mode, VBUS output is turned on. Additionally, the output current of the voltage regulator SGM2590D is determined by the resistor

4. For the corresponding DTS configuration of Type-C to Type-A USB 3.1/DP, please refer to [Type-C to Type-A USB 3.1/DP DTS Configuration](#).

**Note:**

In theory, the hardware circuit can also be designed to use lane2/3 for Type-A USB 3.1 and lane0/1 for DP, but the software needs to modify the property `rockchip,dp-lane-mux` of the usbdp\_phy node to adapt to the hardware design.

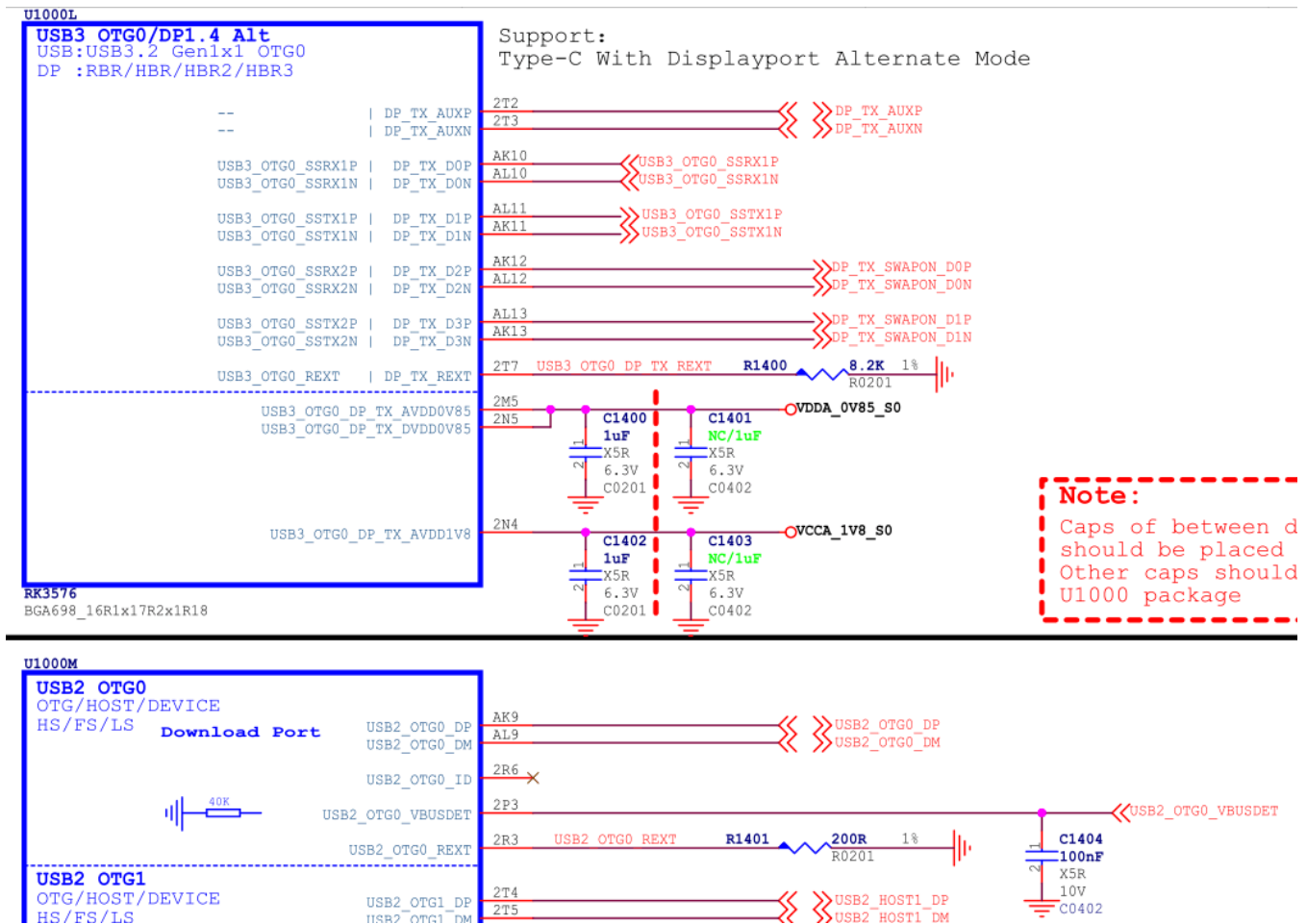


Figure 11 RK3576 Type-A USB 3.1/DP Circuit

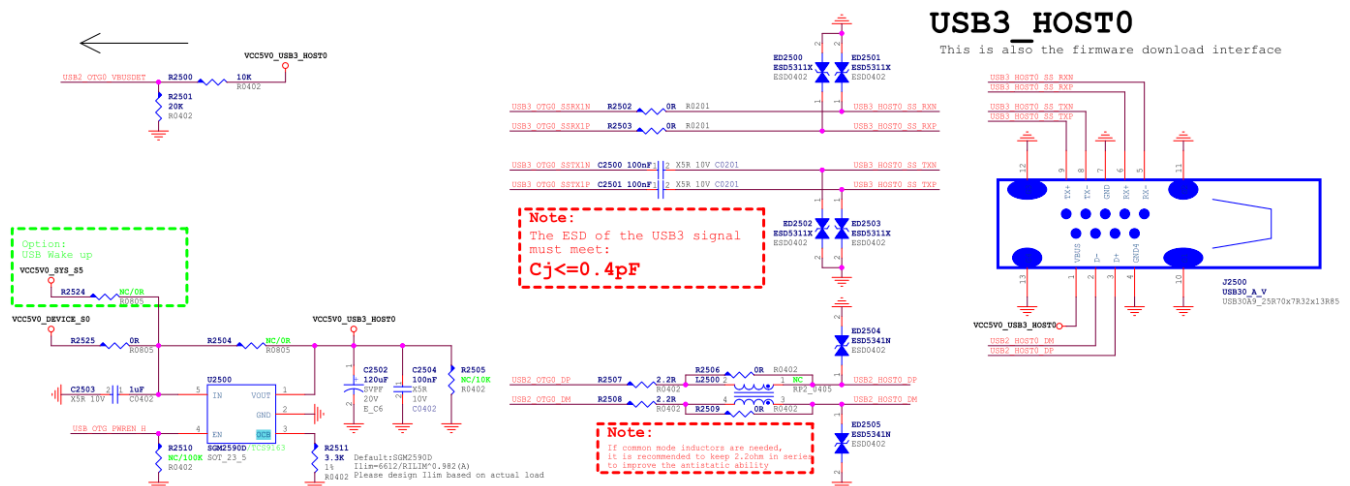


Figure 12 RK3576 Type-A USB3.1 Interface

### 3.3.6 Type-C to Type-A/Micro USB 2.0/DP Hardware Circuit

This solution is applicable for RK3576 USB 3.1 OTG0 Type-C which can be split into an independent Type-A USB 2.0 interface and a DP (4 x Lane) interface. Taking the RK3576 TEST2 Board Type-C to Type-A USB 2.0/DP hardware circuit design as an example.

1. The 4 x Lane of TYPEC USBDP PHY is used entirely for the DP interface, and USB does not use USBDP PHY;
2. When TYPEC USB is used only in HOST mode, USB2\_OTG0\_ID and USB2\_OTG0\_VBUSDET can be left floating;
3. The power supply for Type-A VBUS (VCC5V0\_USB2\_OTG0) is controlled by GPIO. When OTG is in HOST mode, VBUS output is turned on. Additionally, the voltage regulator SGM2590D also has current limiting configuration;
4. For the corresponding DTS configuration of Type-C to Type-A USB 2.0/DP, please refer to [Type-C to Type-A/Micro USB 2.0/DP DTS Configuration](#).

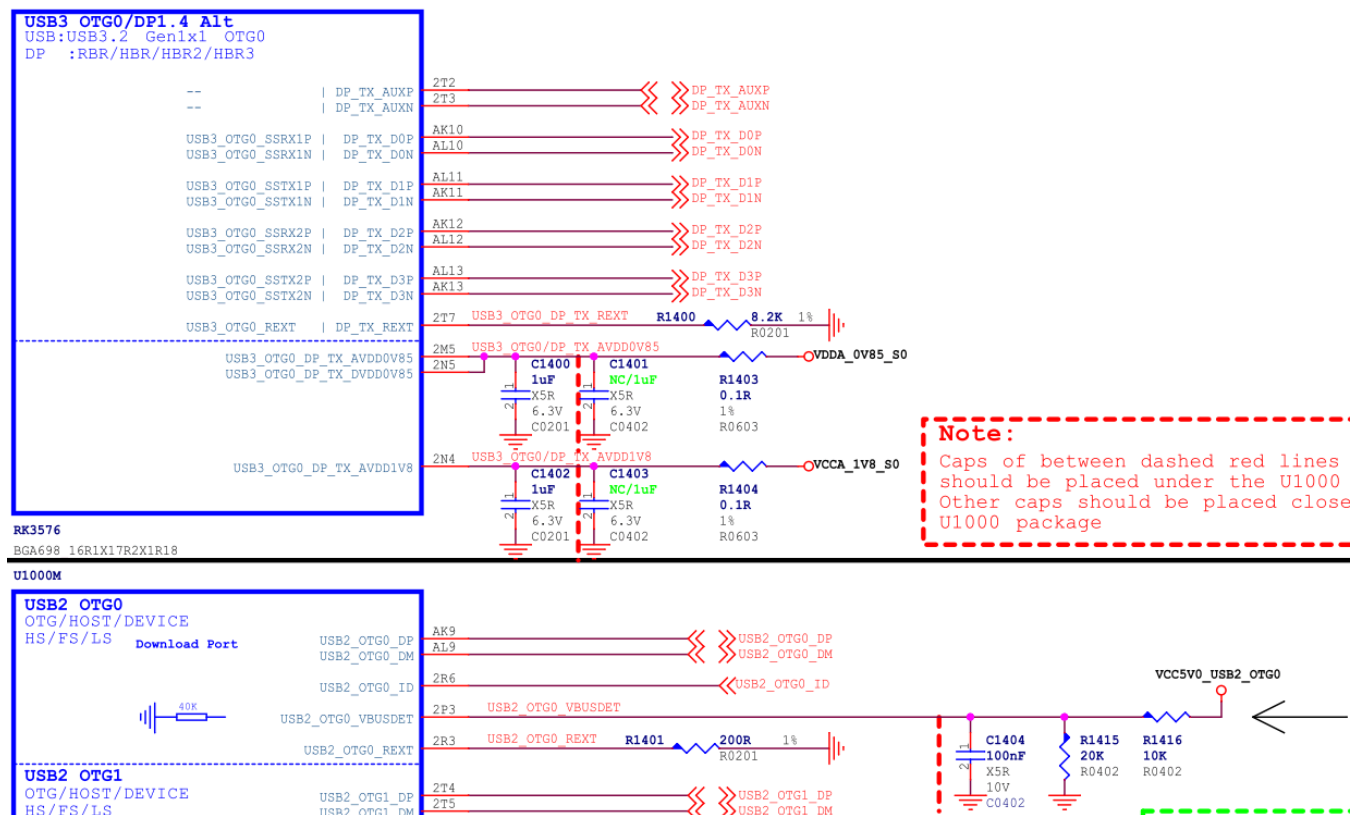


Figure 13 RK3576 Type-A USB 2.0/DP Circuit

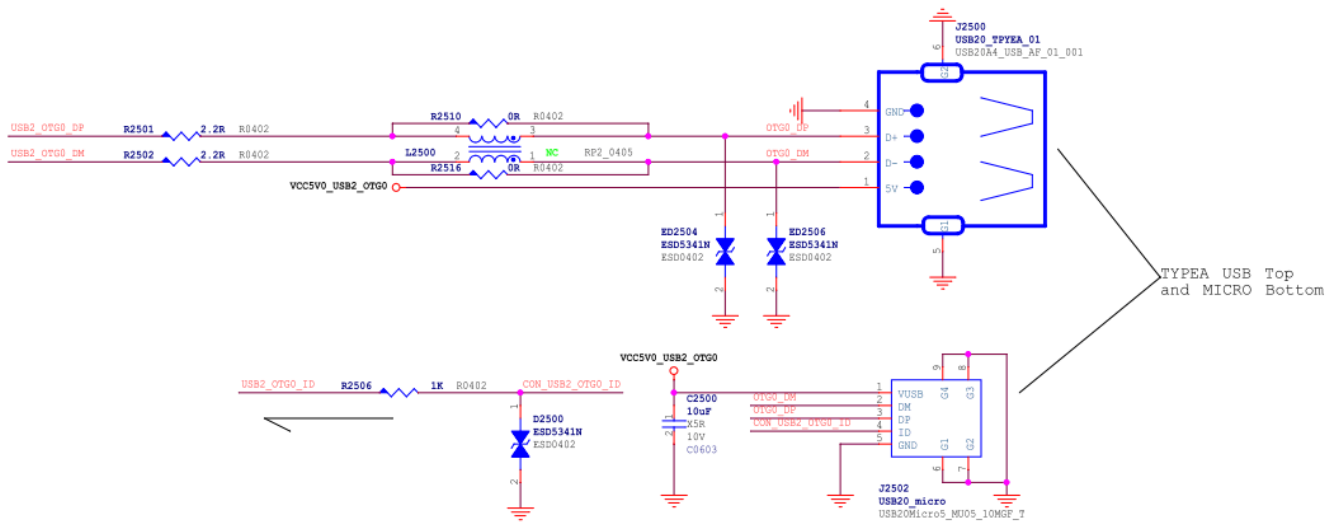


Figure 14 RK3576 Type-A/Micro USB2 Interface

### 3.3.7 Type-A USB3.1 OTG1 Hardware Circuit

As previously mentioned, the key difference between RK3576 USB3.1 OTG0 and OTG1 lies in the connection to different USB3 PHYs. Therefore, the non-Type-C hardware circuit for RK3576 USB 3.1 OTG1 is essentially the same as the USB 3.1 OTG0 USB circuit; for a full-featured Type-C circuit, an additional USB3.1 Switch chip (such as FUSB340) and a Type-C controller (such as FUSB302) are required between OTG1 and the Type-C interface.

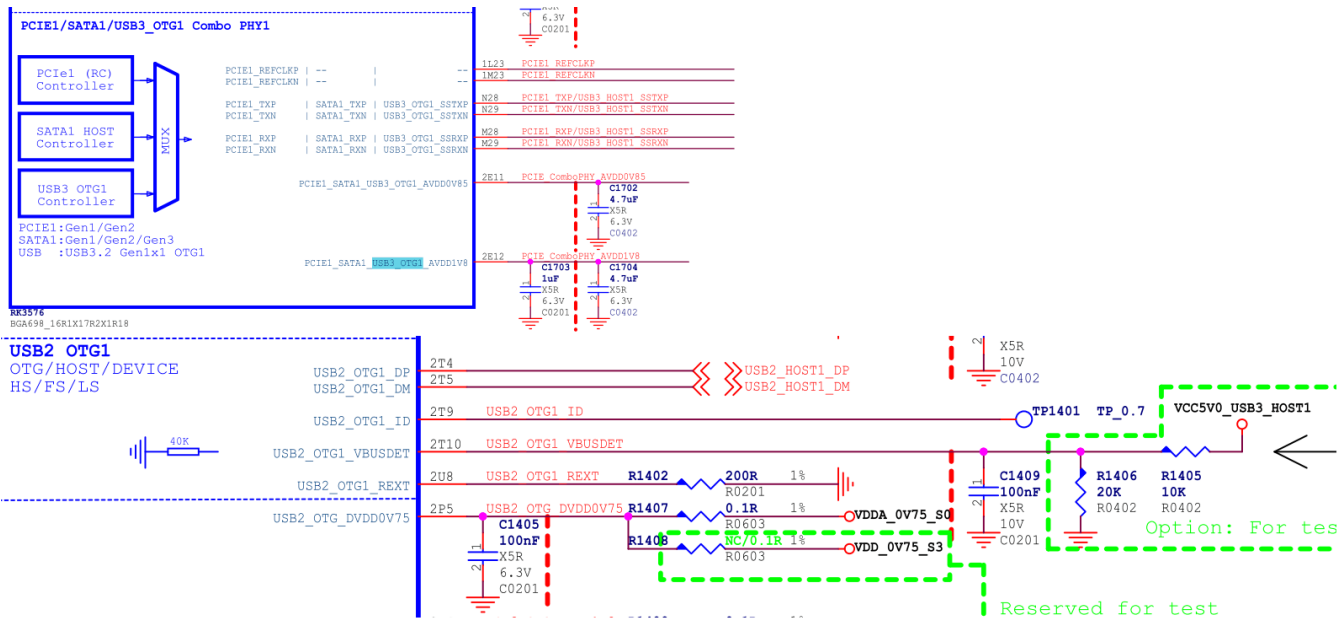


Figure 15 RK3576 USB3.1 OTG1 Interface

## 4. RK3576 USB DTS Configuration

RK3576 USB DTS Configuration includes: chip-level USB controller/PHY DTSI configuration and board-level DTS configuration. For detailed configuration methods, please refer to the following kernel documentation:

1. kernel/Documentation/devicetree/bindings/usb/snps,dwc3.yaml
2. kernel/Documentation/devicetree/bindings/connector/usb-connector.yaml
3. kernel/Documentation/devicetree/bindings/phy/phy-rockchip-inno-usb2.yaml
4. kernel/Documentation/devicetree/bindings/phy/phy-rockchip-usbdp.yaml
5. kernel/Documentation/devicetree/bindings/phy/phy-rockchip-naneng-combphy.txt

## 4.1 USB Chip-Level DTSI Configuration

The main nodes related to the USB controller and PHY in the RK3576 DTSI file are shown in the table below. Since the USB DTSI node configuration involves the common resources and attributes of the USB controller and PHY, it is recommended that developers do not make changes. The complete path of the corresponding DTSI is as follows:

```
arch/arm64/boot/dts/rockchip/RK3576.dtsi
```

Table 9 Correspondence between RK3576 USB Interfaces and USB DTS Nodes

| USB Interface Name (Schematic) | USB Controller DTS Node | USB PHY DTS Node                                  |
|--------------------------------|-------------------------|---------------------------------------------------|
| USB3 OTG0                      | usb_drd0_dwc3           | u2phy0<br>u2phy0_otg<br>usbdp_phy<br>usbdp_phy_u3 |
| USB3 OTG1                      | usb_drd1_dwc3           | u2phy1<br>u2phy1_otg<br>combphy1_psu              |

USB Controller DTSI Nodes are as follows:

```
#USB3.1 OTG0 Controller
usb_drd0_dwc3: usb@23000000 {
    compatible = "rockchip,rk3576-dwc3", "snps,dwc3";
    .....
};

#USB3.1 OTG1 Controller
usb_drd1_dwc3: usb@23400000 {
    compatible = "rockchip,rk3576-dwc3", "snps,dwc3";
    .....
};
```

USB PHY DTSI Nodes are as follows:

Note: USB PHY and USB controllers have a one-to-one correspondence and need to be configured in pairs. For the internal connection relationship between USB PHY and controller in the chip, please refer to [RK3576 USB Controller and PHY Introduction](#) Figure 1 and Table 9. In the DTSI node, the corresponding USB PHY is associated through the "phys" property of the USB controller node.

```

usb2phy_grf: syscon@2602e000 {
    compatible = "rockchip,RK3576-usb2phy-grf", "syscon",
        "simple-mfd";
    .....

    // USB2.0 PHY0
    u2phy0: usb2-phy@0 {
        compatible = "rockchip,RK3576-usb2phy";
        .....
        u2phy0_otg: otg-port {
            #phy-cells = <0>;
            status = "disabled";
        };
    };

    // USB2.0 PHY1
    u2phy1: usb2-phy@2000 {
        compatible = "rockchip,RK3576-usb2phy";
        .....
        u2phy1_otg: otg-port {
            #phy-cells = <0>;
            status = "disabled";
        };
    };
};

#USB3.1/DP Combo PHY
usbdp_phy: phy@2b010000 {
    compatible = "rockchip,rk3576-usbdp-phy";
    .....
    usbdp_phy_dp: dp-port {
        #phy-cells = <0>;
        status = "disabled";
    };

    usbdp_phy_u3: u3-port {
        #phy-cells = <0>;
        status = "disabled";
    };
};

#USB3.1/SATA/PCIe PHY1
combphy1_psu: phy@2b060000 {
    compatible = "rockchip,rk3576-naneng-combphy";
    .....
};

```

## 4.2 USB 3.1/DP Full-Function DTS Configuration

Refer to the DTS configuration of the usbc0 interface in `arch/arm64/boot/dts/rockchip/rk3576-evb1.dtsi`.

```
// USB2.0 PHY Configuration Property "rockchip,typec-vbus-det", indicating support for
// hardware designs with Type-C VBUS_DET constantly pulled high
&u2phy0_otg {
    rockchip,typec-vbus-det;
};

// USB3.1/DP PHY, configure properties "sbu1-dc-gpios" and "sbu2-dc-gpios" according to
// hardware design
&usbdp_phy {
    orientation-switch;
    svid = <0xff01>;
    sbu1-dc-gpios = <&gpio2 RK_PA6 GPIO_ACTIVE_HIGH>;
    sbu2-dc-gpios = <&gpio2 RK_PA7 GPIO_ACTIVE_HIGH>;

    port {
        #address-cells = <1>;
        #size-cells = <0>;
        usbdp_phy_orientation_switch: endpoint@0 {
            reg = <0>;
            remote-endpoint = <&usbc0_orien_sw>;
        };

        usbdp_phy_dp_altmode_mux: endpoint@1 {
            reg = <1>;
            remote-endpoint = <&dp_altmode_mux>;
        };
    };
};

// USB3.1 OTG0 Controller
&usb_drd0_dwc3 {
    dr_mode = "otg";
    usb-role-switch;
    port {
        usb_drd0_role_switch: endpoint {
            remote-endpoint = <&usbc0_role_sw>;
        };
    };
};

// VBUS GPIO Configuration, controlled by the Type-C controller chip driver
vbus5v0_typec: vbus5v0-typec {
    compatible = "regulator-fixed";
    regulator-name = "vbus5v0_typec";
    regulator-min-microvolt = <5000000>;
    regulator-max-microvolt = <5000000>;
    enable-active-high;
    gpio = <&gpio0 RK_PD1 GPIO_ACTIVE_HIGH>;
};
```

```

    vin-supply = <&vcc5v0_device>;
    pinctrl-names = "default";
    pinctrl-0 = <&usb_otg0_pwren>;
};

// Configure external Type-C controller chip HUSB311
// Configure properties "I2C/interrupts/vbus-supply/usb_con" according to actual hardware
design
&i2c2 {
    status = "okay";

    usbc0: husb311@4e {
        compatible = "hynetek,husb311"; // Note 1
        reg = <0x4e>;
        interrupt-parent = <&gpio0>;
        interrupts = <RK_PA5 IRQ_TYPE_LEVEL_LOW>;
        pinctrl-names = "default";
        pinctrl-0 = <&usbc0_int>;
        vbus-supply = <&vbus5v0_typec>;
        status = "okay";

        port {
            usbc0_role_sw: endpoint {
                remote-endpoint = <&usb_drd0_role_switch>;
            };
        };

        usb_con: connector {
            compatible = "usb-c-connector";
            label = "USB-C";
            data-role = "dual";
            power-role = "dual";
            try-power-role = "sink";
            op-sink-microwatt = <1000000>;

            // Configure charging/supplying capability according to actual
hardware design

            sink-pdos =
                <PDO_FIXED(5000, 1000, PDO_FIXED_USB_COMM)>;
            source-pdos =
                <PDO_FIXED(5000, 3000, PDO_FIXED_USB_COMM)>;

            altmodes {
                #address-cells = <1>;
                #size-cells = <0>;

                altmode@0 {
                    reg = <0>;
                    svid = <0xff01>;
                    vdo = <0xffffffff>;
                };
            };
        };
    };
};

```



```

        ports {
            #address-cells = <1>;
            #size-cells = <0>;

            port@0 {
                reg = <0>;
                usbc0_orien_sw: endpoint {
                    remote-endpoint =
<&usbdp_phy_orientation_switch>;
                };
            };

            port@1 {
                reg = <1>;
                dp_altmode_mux: endpoint {
                    remote-endpoint =
<&usbdp_phy_dp_altmode_mux>;
                };
            };
        };
    };
};

```

### Note: Software modifications using FUSB302/AW35615/AW35615 chips as a replacement for HUSB311 chips

1. DTS Modification: Simple modifications based on the DTS configuration of HUSB311 are sufficient, as shown below:

```

// Configure external Type-C controller chip HUSB311
&i2c2 {
    usbc0: fusb302@22 {
        compatible = "fcs,fusb302";
        reg = <0x22>;
        ..... // The rest is the same as HUSB311
    };
};

```

2. Driver Software Modification: Modify the irq wakeup mechanism of the FUSB302 driver to support various application scenarios such as standby VBUS power off or supply.

```

diff --git a/drivers/usb/typec/tcpm/fusb302.c b/drivers/usb/typec/tcpm/fusb302.c
index 77a050cd2e88..72bef94887b4 100644
--- a/drivers/usb/typec/tcpm/fusb302.c
+++ b/drivers/usb/typec/tcpm/fusb302.c
@@ -105,6 +105,7 @@ struct fusb302_chip {
    bool vbus_on;
    bool charge_on;
    bool vbus_present;
+   bool wakeup;
    enum typec_cc_polarity cc_polarity;

```

```

enum typec_cc_status cc1;
enum typec_cc_status cc2;
@@ -1769,7 +1770,8 @@ static int fusb302_probe(struct i2c_client *client,
    dev_err(dev, "cannot request IRQ for GPIO Int_N, ret=%d", ret);
    goto tcpm_unregister_port;
}
- enable_irq_wake(chip->gpio_int_n_irq);
+ chip->wakeup = device_property_read_bool(dev, "wakeup-source");
+ device_init_wakeup(dev, true);
i2c_set_clientdata(client, chip);

return ret;
@@ -1788,7 +1790,7 @@ static void fusb302_remove(struct i2c_client *client)
{
    struct fusb302_chip *chip = i2c_get_clientdata(client);

- disable_irq_wake(chip->gpio_int_n_irq);
+ device_init_wakeup(chip->dev, false);
free_irq(chip->gpio_int_n_irq, chip);
kthread_destroy_worker(chip->irq_worker);
cancel_delayed_work_sync(&chip->bc_lvl_handler);
@@ -1809,6 +1811,11 @@ static int fusb302_pm_suspend(struct device *dev)

    /* Make sure any pending irq_work is finished before the bus suspends */
    kthread_flush_worker(chip->irq_worker);
+
+ if (device_may_wakeup(dev) && (!chip->vbus_on || chip->wakeup))
+     enable_irq_wake(chip->gpio_int_n_irq);
+ else
+     disable_irq(chip->gpio_int_n_irq);
    return 0;
}

@@ -1819,6 +1826,11 @@ static int fusb302_pm_resume(struct device *dev)
    u8 pwr;
    int ret = 0;

+ if (device_may_wakeup(dev) && (!chip->vbus_on || chip->wakeup))
+     disable_irq_wake(chip->gpio_int_n_irq);
+ else
+     enable_irq(chip->gpio_int_n_irq);
+
    /*
     * When the power of fusb302 is lost or i2c read failed in PM S/R
     * process, we must reset the tcpm port first to ensure the devices

```

## 4.3 USB 3.1/DP DTS Configuration for Type-C to Type-A

Refer to `arch/arm64/boot/dts/rockchip/RK3576-evb2.dtsi` for the DTS configuration of Type-C to Type-A USB 3.1/DP.

```
#USB2.0 PHY configuration for "phy-supply" attribute, used to control VBUS output at 5V
&u2phy0_otg {
    phy-supply = <&vcc5v0_otg>;
};

#VBUS GPIO configuration, controlled by this GPIO in the USB2.0 PHY driver
vcc5v0_otg: vcc5v0-otg {
    compatible = "regulator-fixed";
    regulator-name = "vcc5v0_otg";
    regulator-min-microvolt = <5000000>;
    regulator-max-microvolt = <5000000>;
    enable-active-high;
    gpio = <&gpio0 RK_PD1 GPIO_ACTIVE_HIGH>;
    vin-supply = <&vcc5v0_device>;
    pinctrl-names = "default";
    pinctrl-0 = <&usb_otg0_pwren>;
};

#USB3.1/DP PHY0, configure DP to use lane2/3, the driver will automatically allocate lane0/1
for USB3.1 Rx/Tx
#If the hardware design uses DP on lane0/1, then "rockchip,dp-lane-mux = <0 1>" should be
configured here
#Note: Even if DP is not supported in the actual circuit, "rockchip,dp-lane-mux" must be
configured, otherwise the USBDP PHY driver cannot automatically allocate lanes to USB3.1
&usbdp_phy {
    rockchip,dp-lane-mux = <2 3>;
};

#USB3.1 OTG0 Controller
#Set "dr_mode" to "otg" and configure the "extcon" attribute to support software switching
between Device/Host mode
&usb_drd0_dwc3 {
    dr_mode = "otg";
    extcon = <&u2phy0>;
    status = "okay";
};
```

## 4.4 Type-C to Type-A/Micro USB 2.0/DP DTS Configuration

Refer to `arch/arm64/boot/dts/rockchip/RK3576-test2.dtsi` for the DTS configuration of Type-C to Type-A USB 2.0/DP.

```
# USB2.0 PHY0 Configuration
# Configure the property "phy-supply" to control the VBUS output of 5V
```

```

# Configure the property "rockchip,sel-pipe-phystatus", indicating the selection of GRF to
control the pipe phystatus, replacing the control of USBDP PHY
# Configure the property "rockchip,dis-u2-susphy", where otg mode is an option, indicating
the function to close the USB2 PHY driver dynamically entering suspend mode is optional
&u2phy0_otg {
    vbus-supply = <&vcc5v0_otg>;
    rockchip,sel-pipe-phystatus;
    rockchip,dis-u2-susphy;
    status = "okay";
};

# VBUS GPIO Configuration, controlling this GPIO in the USB2.0 PHY driver
vcc5v0_otg: vcc5v0-otg {
    ...
};

# USB3.1/DP PHY, configure DP to use lanes 0/1/2/3
# According to the actual hardware design, configure the property "rockchip,dp-lane-mux"
# Configure the property maximum-speed = "high-speed", to notify the USBDP PHY driver to
restrict USB to USB2.0 only
&usbdp_phy {
    maximum-speed = "high-speed";
    rockchip,dp-lane-mux = < 0 1 2 3 >;
    status = "okay";
};

&usbdp_phy_dp {
    status = "okay";
};

&usbdp_phy_u3 {
    status = "okay";
};

# Configure the property "maximum-speed", to notify the DWC3 driver to restrict USB to
USB2.0 only
# Configure the property "snps,dis_u2_susphy_quirk", to disable the controller hardware's
suspend usb2 phy function, improving the stability of USB communication
# Configure the property "snps,usb2-lpm-disable", to disable the controller's Host mode LPM
function, improving the compatibility of USB peripherals
&usb_drd0_dwc3 {
    phys = <&u2phy0_otg>;
    phy-names = "usb2-phy";
    extcon = <&u2phy0>;
    maximum-speed = "high-speed";
    snps,dis_u2_susphy_quirk;
    snps,usb2-lpm-disable;
    status = "okay";
};

```

## 4.5 Type-C USB 2.0 only DTS Configuration

Configuration 1. Hardware circuit integrates an external Type-C controller chip, supporting PD

```
#USB2.0 PHY0 registers typec orientation switch, used for interaction with the TCPM
subsystem, to obtain USB hot-plug information
&u2phy0 {
    orientation-switch;
    status = "okay";

    port {
        u2phy0_orientation_switch: endpoint {
            remote-endpoint = <&usbc0_orien_sw>;
        };
    };
};

#USB2.0 PHY0 OTG configuration
#Configuration property "rockchip,sel-pipe-phystatus", indicates the selection of GRF
control pipe phystatus, replacing the control of USBDP PHY
#Configuration property "rockchip,typec-vbus-det", indicates support for Type-C VBUS_DET
hardware designs that are always pulled high
#Configuration property "rockchip,dis-u2-susphy", optional, indicates disabling the USB2 PHY
driver's dynamic entry into suspend mode function
&u2phy0_otg {
    rockchip,sel-pipe-phystatus;
    rockchip,typec-vbus-det;
    rockchip,dis-u2-susphy;
    status = "okay";
};

#Configure the USBDP CombPHY parent node
#Note: The usbdp_phy node must be correctly configured according to whether the actual
hardware circuit USBDP CombPHY is powered.
#Case 1. If the actual hardware circuit CombPHY is designed to be powered, then disable all
relevant parent nodes and child nodes of usbdp_phy, keeping the USBDP PHY in an
uninitialized state to achieve the lowest power consumption
&usbdp_phy {
    status = "disabled";
};

#Case 2. If the actual hardware circuit CombPHY is not powered, then enable the usbdp_phy
node and disable related child nodes to prevent system startup exceptions or USB download
function exceptions
&usbdp_phy {
    rockchip,usbdpphy-clamp; /* To avoid power-off affecting bus access and Logic
leakage to USBDP PHY */
    maximum-speed = "high-speed"; /* Limit u2 only, to avoid initializing USBDP PHY */
    status = "okay";
};

#Configure the USBDP CombPHY child nodes
&usbdp_phy_dp {
```

```

        status = "disabled";
};

&usbdp_phy_u3 {
    status = "disabled";
};

#Configure the DP controller
&dp {
    status = "disabled";
};

#Configure the USB3.1 OTG0 Controller
#Configure "phys = <&u2phy0_otg>", which means not to reference the USBDP PHY
#Configure maximum-speed = "high-speed", to inform the DWC3 driver to limit USB to USB2.0
only
#Configuration property "snps,dis_u2_susphy_quirk", to disable the controller's hardware
suspend usb2 phy function, improving the stability of USB communication
#Configuration property "snps,usb2-lpm-disable", to disable the controller's Host mode LPM
function, improving the compatibility of USB peripherals
&usb_drd0_dwc3 {
    dr_mode = "otg";
    status = "okay";
    maximum-speed = "high-speed";
    phys = <&u2phy0_otg>;
    phy-names = "usb2-phy";
    usb-role-switch;
    snps,dis_u2_susphy_quirk;
    snps,usb2-lpm-disable;
    port {
        usb_drd0_role_switch: endpoint {
            remote-endpoint = <&usbc0_role_sw>;
        };
    };
};

#Configure the external Type-C controller chip HUSB311
#Properties "I2C/interrupts/vbus-supply/usb_con" must be configured according to the actual
hardware design
#The property usbc0_orien_sw's remote-endpoint must be set to <&u2phy0_orientation_switch>
&i2c2 {
    status = "okay";
    usbc0: husb311@4e {
        compatible = "hynetek,husb311";
        reg = <0x4e>;
        interrupt-parent = <&gpio0>;
        interrupts = <RK_PC4 IRQ_TYPE_LEVEL_LOW>;
        pinctrl-names = "default";
        pinctrl-0 = <&usbc0_int>;
        vbus-supply = <&vbus5v0_typec>;
        status = "okay";

        port {

```

```

        usbc0_role_sw: endpoint {
            remote-endpoint = <&usb_drd0_role_switch>;
        };
    };

    usb_con: connector {
        compatible = "usb-c-connector";
        label = "USB-C";
        data-role = "dual";
        power-role = "dual";
        .....
        port {
            usbc0_orien_sw: endpoint {
                remote-endpoint = <&u2phy0_orientation_switch>;
            };
        };
    };
};

```

Configuration 2. Hardware circuit without an external Type-C controller chip, supporting Device only  
Type-C USB 2.0 Device's DTS Configuration

```

#USB2.0 PHY0 OTG configuration
#Configuration property "rockchip,sel-pipe-phystatus", indicates the selection of GRF
control pipe phystatus, replacing the control of USBDP PHY
#Configuration property "rockchip,dis-u2-susphy", a required option, indicates disabling the
USB2 PHY driver's dynamic entry into suspend mode function
&u2phy0_otg {
    rockchip,sel-pipe-phystatus;
    rockchip,dis-u2-susphy;
    status = "okay";
};

#Configure the USBDP CombPHY parent node
#Note: The usbdp_phy node must be correctly configured according to whether the actual
hardware circuit USBDP CombPHY is powered.
#Case 1. If the actual hardware circuit CombPHY is designed to be powered, then disable all
relevant parent nodes and child nodes of usbdp_phy, keeping the USBDP PHY in an
uninitialized state to achieve the lowest power consumption
&usbdp_phy {
    status = "disabled";
};

#Case 2. If the actual hardware circuit CombPHY is not powered, then enable the usbdp_phy
node and disable related child nodes to prevent system startup exceptions or USB download
function exceptions
&usbdp_phy {
    rockchip,usbdpphy-clamp; /* To avoid power-off affecting bus access and Logic
leakage to USBDP PHY */
    maximum-speed = "high-speed"; /* Limit u2 only, to avoid initializing USBDP PHY */
    status = "okay";
};

```

```

#Configure the USBDP CombPHY child nodes
&usbdp_phy_dp {
    status = "disabled";
};

&usbdp_phy_u3 {
    status = "disabled";
};

#Configure the DP controller
&dp {
    status = "disabled";
};

#Configure the USB3.1 OTG0 Controller
#Configure dr_mode = "peripheral", to inform the DWC3 driver to initialize in Device only
mode
#Configure "phys = <&u2phy0_otg>", which means not to reference the USBDP PHY
#Configure maximum-speed = "high-speed", to inform the DWC3 driver to limit USB to USB2.0
only
#Configuration property "snps,dis_u2_susphy_quirk", to disable the controller's hardware
suspend usb2 phy function, improving the stability of USB communication
&usb_drd0_dwc3 {
    dr_mode = "peripheral";
    phys = <&u2phy0_otg>;
    phy-names = "usb2-phy";
    maximum-speed = "high-speed";
    snps,dis_u2_susphy_quirk;
    status = "okay";
};

```

**Configuration 3. Hardware circuit without an external Type-C controller chip, supporting OTG (CC to ID level conversion circuit is required)**

```

#USB2.0 PHY0 OTG configuration
#Configuration property "rockchip,sel-pipe-phystatus", indicates the selection of GRF
control pipe phystatus, replacing the control of USBDP PHY
#Configuration property "rockchip,dis-u2-susphy", otg mode is an optional, indicates
disabling the USB2 PHY driver's dynamic entry into suspend mode function
&u2phy0_otg {
    rockchip,sel-pipe-phystatus;
    rockchip,dis-u2-susphy;
    status = "okay";
};

#Configure the USBDP CombPHY parent node
#Note: The usbdp_phy node must be correctly configured according to whether the actual
hardware circuit USBDP CombPHY is powered.
#Case 1. If the actual hardware circuit CombPHY is designed to be powered, then disable the
parent node and child nodes of usbdp_phy, keeping the USBDP PHY in an uninitialized state to
achieve the lowest power consumption
&usbdp_phy {

```



```

        status = "disabled";
};

#Case 2. If the actual hardware circuit CombPHY is not powered, then enable the usbdp_phy
node and disable related child nodes to prevent system startup exceptions or USB download
function exceptions
&usbdp_phy {
    rockchip,usbdpphy-clamp; /* To avoid power-off affecting bus access and Logic
leakage to USB DP PHY */
    maximum-speed = "high-speed"; /* Limit u2 only, to avoid initializing USB DP PHY */
    status = "okay";
};

#Configure the USB DP CombPHY child nodes
&usbdp_phy_dp {
    status = "disabled";
};

&usbdp_phy_u3 {
    status = "disabled";
};

#Configure the DP controller
&dp {
    status = "disabled";
};

#Configure the USB3.1 OTG0 Controller
#Configure dr_mode = "otg"
#Configure "phys = <&u2phy0_otg>", which means not to reference the USB DP PHY
#Configure maximum-speed = "high-speed", to inform the DWC3 driver to limit USB to USB2.0
only
#Configure "extcon" property, to support automatic switching between Device/Host mode
#Configuration property "snps,dis_u2_susphy_quirk", to disable the controller's hardware
suspend usb2 phy function, improving the stability of USB communication
#Configuration property "snps,usb2-lpm-disable", to disable the controller's Host mode LPM
function, improving the compatibility of USB peripherals
&usb_drd0_dwc3 {
    dr_mode = "otg";
    phys = <&u2phy0_otg>;
    phy-names = "usb2-phy";
    maximum-speed = "high-speed";
    extcon = <&u2phy0>;
    snps,dis_u2_susphy_quirk;
    snps,usb2-lpm-disable;
    status = "okay";
};

```

## 4.6 USB 3.1 DTS Configuration for Type-A

Refer to `arch/arm64/boot/dts/rockchip/rk3576-evb1.dtsi` for USB3 OTG1 DTS configuration.

```
#USB2.0 PHY1 configuration for "phy-supply" attribute to control VBUS output at 5V
&u2phy1_otg {
    phy-supply = <&vcc5v0_host>;
}

#VBUS GPIO configuration to control this GPIO in the USB2.0 PHY driver
vcc5v0_host: vcc5v0-host {
    compatible = "regulator-fixed";
    regulator-name = "vcc5v0_host";
    regulator-boot-on;
    regulator-always-on;
    regulator-min-microvolt = <5000000>;
    regulator-max-microvolt = <5000000>;
    enable-active-high;
    gpio = <&gpio0 RK_PC7 GPIO_ACTIVE_HIGH>;
    vin-supply = <&vcc5v0_device>;
    pinctrl-names = "default";
    pinctrl-0 = <&vcc5v0_host_en>;
};

#Enable USB3.1/SATA/PCIe Combo PHY
&combphy1_psu {
    status = "okay";
};

#Configure USB3.1 OTG1 Controller
&usb_drd1_dwc3 {
    dr_mode = "host";
    status = "okay";
};
```

## 4.7 DTS Configuration for USB PHY Without Power Supply

The external power supply for USBDP PHY (OTG0 USB3.1 and DP multiplexing) and ComboPHY1 (OTG1 USB3.1 and PCIe1/SATA1 multiplexing) can be allowed to be unpowered to reduce system operational power consumption and simplify hardware circuit design. However, it is necessary to modify the corresponding DTS configuration in software to avoid system startup exceptions or USB functionality abnormalities.

### 4.7.1 USBDP PHY DTS Configuration Without Power Supply

```
&usbdp_phy {
    rockchip,usbdpphy-clamp; /* Prevent power-off from affecting bus access and Logic
leakage to USBDP PHY */
    maximum-speed = "high-speed"; /* Restrict u2 only, to avoid initializing USBDP PHY
*/
    status = "okay";
};

&usbdp_phy_dp {
    status = "disabled";
};

&usbdp_phy_u3 {
    status = "disabled";
};
```

#### Kernel Activation Log

```
rockchip-usbdp-phy 2b010000.phy: Failed to enable usbdpphy because clamp is set
rockchip-usbdp-phy: probe of 2b010000.phy failed with error -95
```

Furthermore, the flashing tool must disable the USB3 download function; otherwise, there will be a problem with the Maskrom downloading firmware failure. The modification method is as follows:

For Windows tool config.ini modification: USB3\_TRANSFER=FALSE

For Linux tool config.ini modification: usb3\_transfer\_on=false

### 4.7.2 ComboPHY1 Powerless DTS Configuration

ComboPHY1 is used for OTG1's USB 3.1, but it also affects OTG1's USB2 access to the SoC bus. Therefore, DTS configuration should be performed based on whether OTG1 USB2.0 is in use.

1. OTG1 USB 3.1 is not used, but USB 2.0 functionality is required

```
&combphy1_psu {
    rockchip,dis-u3otg1-port;
    status = "okay";
};

&usb_drd1_dwc3 {
    dr_mode = "host";
    phys = <&u2phy1_otg>, <&combphy1_psu PHY_TYPE_USB3>; /* This must reference
combphy1_psu */
    phy-names = "usb2-phy", "usb3-phy";
    maximum-speed = "high-speed";
    snps,dis_u2_susphy_quirk;
    snps,usb2-lpm-disable;
```

```
        status = "okay";  
};
```

2. OTG1 USB 2.0 and USB 3.1 are both not used

```
&combphy1_psu {  
    status = "disabled";  
};  
  
&usb_drd1_dwc3 {  
    status = "disabled";  
};
```

## 4.8 Considerations for Configuring Linux USB DT

### 4.8.1 Important Properties of USB DT

#### 4.8.1.1 USB Controller Attributes

1. The "usb-role-switch" is exclusively used for standard Type-C interfaces equipped with a PD controller chip, and the dr\_mode attribute must be configured as "otg"; if dr\_mode is not in "otg" mode, do not configure the "usb-role-switch";
2. The primary function of the "extcon" attribute is to dynamically switch the OTG mode, suitable for hardware circuit designs of non-standard Type-C interfaces (equipped with a PD controller chip), such as USB Micro interfaces or Type-A interfaces, and in schemes where the controller is configured in "otg" mode, to achieve dynamic switching of the OTG mode;
3. "snps,dis\_u2\_susphy\_quirk", disables the USB controller hardware's automatic suspend function for the usb2 phy, mainly used in USB 2.0 only solutions to enhance the stability of USB communication;
4. "snps,usb2-lpm-disable", disables the LPM function of the USB controller in Host mode, mainly used in USB 2.0 only solutions to improve the compatibility of USB peripherals.

#### 4.8.1.2 USB2 PHY Attributes

1. "vbus-supply" and "phy-supply" are both used to control the 5V VBUS output, but they have different usage scenarios. "vbus-supply" is used for the OTG port, supporting dynamic switching of VBUS. "phy-supply" is used for the USB HOST port, where VBUS 5V is always on after the system powers up;
2. "rockchip,sel-pipe-phystatus", this attribute is used to configure the GRF USB control register, selecting GRF control pipe phystatus, replacing the control of the USB DP PHY. It is mainly used in USB 2.0 only solutions. If the USB DP PHY is not enabled, this attribute must be added; otherwise, the USB device cannot work properly;
3. "rockchip,typec-vbus-det", used to support hardware designs where Type-C VBUS\_DET is always pulled high;
4. "rockchip,dis-u2-susphy", disables the USB2 PHY driver's dynamic entry into suspend mode, mainly

used in USB 2.0 only solutions, to maintain the USB2 PHY output clock to the USB controller.

#### 4.8.1.3 USBDP Combo PHY Properties

1. "rockchip,dp-lane-mux", in non-full-featured Type-C solutions, configures the Lane number for DP mapping. DP supports 2 or 4 lanes, for example, "rockchip,dp-lane-mux = <2, 3>," indicates that DP Lane0 is mapped to Lane2 of the USBDP PHY, and DP Lane1 is mapped to Lane3 of the USBDP PHY; similarly, "rockchip,dp-lane-mux = <0, 1, 2, 3>," indicates that DP Lane0 is mapped to Lane0 of the USBDP PHY, and so on, and so forth.

Note: In actual circuitry, if only USB3.1 is supported but not DP, "rockchip,dp-lane-mux" must still be configured; otherwise, the USBDP PHY driver cannot automatically allocate lanes to USB3.1.

2. maximum-speed = "high-speed"; this property limits USB to USB2.0 only within the USBDP PHY driver, note that this property is configured within the parent node usbdp\_phy.

#### 4.8.1.4 USBC Properties

1. The `usbc` node typically acts as a child node of `i2c`, and in addition to basic properties such as `compatible` and `reg`, it also includes two child nodes: `port` and `connector`. The `port` node is used to configure the `role switch` property that interacts with the USB controller; the `connector` node, apart from configuring the basic properties of PD, also includes `altmodes` and `ports` (used to configure `orientation switch` and `altmode mux`) 2 child nodes.
2. In the `connector` node, the `sink-pdos` and `source-pdos` properties are used to configure the voltage/current levels and other power capabilities supported by RK3576 when acting as a Sink and Source, respectively. It is important to note that the voltage/current levels must be consistent with the capabilities of the charging IC. The fields of the `sink-pdos` and `source-pdos` properties must adhere to the PD Spec, and key fields must use the macros defined in the following file.

```
include/dt-bindings/usb/pd.h
```

## 5. RK3576 USB OTG Mode Switching Commands

The RK3576 SDK supports forcing the USB OTG to switch to Host or Peripheral mode through software methods, independent of the USB hardware circuit's OTG ID pin or Type-C interface.

There are two methods to switch the RK3576 Linux-6.1 kernel:

Note: Method 1 relies on the correct configuration of the USB DTS and is only applicable to hardware circuit designs without Type-C interfaces. Method 2 has no restrictions. Therefore, when unsure about the correct adaptation of hardware and software, Method 2 is recommended.

Method 1. [Legacy]

```
#1.Force host mode
echo host > /sys/devices/platform/2602e000.syscon/2602e000.syscon:usb2-phy@0/otg_mode
#2.Force peripheral mode
echo peripheral > /sys/devices/platform/2602e000.syscon/2602e000.syscon:usb2-phy@0/otg_mode
```

Method 2. [New]

```
#1.Force host mode
echo host > /sys/kernel/debug/usb/230000000.usb/mode
#2.Force peripheral mode
echo device > /sys/kernel/debug/usb/230000000.usb/mode
```

## 6. Type-C Controller Chip Support List

Table 17 Type-C Controller Chip Support List

| Type-C Controller Chip Model | Linux-6.1   | Description                                                                                                                       |
|------------------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------|
| ANX7411                      | In Testing  | Software driver is supported, functionality to be further tested                                                                  |
| AW35615                      | Supported   | Recommended for priority use, good compatibility with DisplayPort Alt Mode<br>Fully compatible with hardware and software FUSB302 |
| ET7301B                      | Supported   | Fully compatible with hardware and software FUSB302 <sup>note1</sup>                                                              |
| ET7303                       | Supported   | Hardware compatible with FUSB302, software driver highly similar to RT1711 <sup>note2</sup>                                       |
| FUSB302                      | Supported   | Common chip used in RK platform <sup>note2</sup>                                                                                  |
| HUSB311                      | Supported   | RK3576 EVB default chip used<br>Hardware compatible with FUSB302, but software driver is not compatible <sup>note3</sup>          |
| RT1711H                      | Supported   | Hardware compatible with FUSB302, software driver highly similar to ET7303 <sup>note4</sup>                                       |
| WUSB3801                     | Unsupported | Custom single-wire communication mechanism, high error rate, cannot guarantee stable communication.                               |

note1.

- Linux-6.1 adopts the TCPM software framework and TCPCI protocol, which theoretically can be

compatible with all Type-C controller chips designed based on the TCPCI standard (e.g., ET7303/HUSB311/RT1711H);

note2.

- The small-packaged ET7303 (it is understood that the original factory does not currently provide a large package) has been verified through the RK platform. The kernel needs to enable CONFIG\_TYPEC\_ET7303 separately.
- For detailed DTS configuration, please refer to the section [Type-C USB 3.1/DP Full Function DTS Configuration](#) for the usbc0 node, only the node name, reg address, and compatible attributes need to be modified.

note3.

- FUSB302 can be directly replaced with HUSB311. The kernel needs to use CONFIG\_TYPEC\_HUSB311 separately, and the DTS configuration points to note are the same as the above note2.

note4.

- RT1711H is hardware compatible with FUSB302/ET7303, and the software driver is highly similar to ET7303. The kernel needs to enable CONFIG\_TYPEC\_RT1711H separately, and the DTS configuration points to note are the same as the above note2.

## 7. Reference Documents

---

1. Universal Serial Bus Type-C Cable and Connector Specification
2. Universal Serial Bus Power Delivery Specification
3. Rockchip Developer Guide USB EN
4. Rockchip RK3576 Technical Reference Manual
5. RK3576 EVB1/EVB2/TEST1/TEST2 Hardware Schematic Diagram